

How to Build This 3D Portfolio

Step-by-step guide — React + Three.js + Vite

Overview

This portfolio combines React 18 with Three.js (via React Three Fiber) to create an immersive 3D experience. The build tool is Vite 5 for fast development and optimized production bundles.

Step 1: Project Setup

1.1 Create the Vite + React project:

```
npm create vite@latest portfolio -- --template react
cd portfolio
npm install
```

1.2 Install dependencies:

```
npm install three @react-three/fiber @react-three/drei @react-three/postprocessing
npm install react-icons react-intersection-observer
```

These packages add: Three.js 3D engine, React bindings for Three.js (R3F), helpers like Float/Stars, bloom post-processing, icon library, and scroll-intersection detection.

Step 2: Project Structure

```
portfolio/
├── index.html          ← SEO tags, fonts, favicon
```

```
├─ vite.config.js      ← Build config, code splitting
├─ src/
│   ├─ main.jsx        ← React entry point
│   ├─ App.jsx         ← Root component with loader
│   ├─ styles/global.css ← CSS variables, animations
│   └─ components/
│       ├─ Navbar.jsx   ← Fixed top nav, mobile menu
│       ├─ Hero.jsx     ← 3D particle field + heading
│       ├─ About.jsx    ← 3D sphere + experience timeline
│       ├─ Skills.jsx   ← 3D orbs + skill bars
│       ├─ Projects.jsx ← Project cards with hover
│       ├─ Contact.jsx  ← Form + social links
│       ├─ Footer.jsx   ← Credits + social icons
│       ├─ Loader.jsx   ← Animated loading screen
│       └─ ErrorBoundary.jsx ← Crash fallback
```

Step 3: Core Components Explained

3.1 Navbar.jsx

A fixed navigation bar (72px tall) that:

- Turns more opaque on scroll using a `requestAnimationFrame`-throttled scroll listener
- Has a mobile hamburger menu (hidden on desktop via CSS `@media`)
- Uses `backdrop-filter: blur()` for the glass-morphism effect

Key technique: The scroll handler uses `requestAnimationFrame` to avoid firing on every scroll pixel — only updates state once per frame.

3.2 Hero.jsx

The hero section with a full-viewport 3D background:

- **ParticleField** — 800 points with random HSL colors, slowly rotating via `useFrame`
- **FloatingGeometry** — 3 wireframe shapes (icosahedron, octahedron, torus) with Float animation
- **RotatingTorusKnot** — A torus knot that spins on two axes
- **Stars** — Background star field from drei

- Content overlays the 3D scene with `position: absolute` and proper z-index

Key technique: The 3D canvas fills the background while content sits on top using absolute positioning. `paddingTop: NAV_HEIGHT + 40` prevents navbar overlap. All 3D components are wrapped in `memo` to avoid unnecessary re-renders.

3.3 About.jsx

Two-column layout with:

- **3D Sphere** — A distorted sphere using `MeshDistortMaterial` that reacts to hover (scales up via ref, not state)
- **Tech Stack Tags** — Colored tag badges
- **Timeline** — Vertical timeline with gradient line and animated entries

Key technique: Hover effects use `mesh.current.scale.setScalar()` directly instead of React `useState`. This avoids re-rendering the Three.js scene on every hover — a major performance win.

3.4 Skills.jsx

Shows technical skills in two ways:

- **3D Orbs** — 6 floating dodecahedrons with wireframe materials, each representing a skill, rotating via `useFrame`
- **Skill Bars** — Categorized progress bars that animate to their percentage when scrolled into view

Uses `useInView` from `react-intersection-observer` to trigger animations when the section becomes visible. The orbs are memoized with `useMemo` to keep positions stable.

3.5 Projects.jsx

6 project cards in a responsive grid:

- Each card has a gradient header, macOS-style traffic light dots, a project image or acronym placeholder
- Hover effects (lift, border glow, image zoom) are done via `useRef` + direct DOM manipulation — no React state

- GitHub and Live buttons are anchor tags that open in new tabs

3.6 Contact.jsx

Two-column layout with contact info and form:

- **Form** — Uses `fetch` to POST to Web3Forms API with your access key
- **States** — Loading spinner, success banner (green), error banner (red), all auto-dismiss
- **Social Links** — GitHub and LinkedIn icons with hover effects

Step 4: Performance Optimizations

4.1 Code Splitting

In `vite.config.js`, Three.js and R3F are split into separate chunks:

```
manualChunks: {  
  three: ['three'],  
  r3f: ['@react-three/fiber', '@react-three/drei', '@react-three/postprocessing'],  
}
```

This prevents a single 1MB+ bundle — instead you get three smaller cacheable chunks.

4.2 Render Optimization

- All 3D hover effects use refs, not state (avoids React re-renders in the R3F loop)
- Particle positions/colors are computed once with `useMemo([])`
- 3D components are wrapped in `React.memo`
- Event handlers use `useCallback` to maintain referential stability

4.3 CSS Performance

- `content-visibility: auto` on sections — browser skips rendering offscreen content
- `will-change` hints on animated elements
- `prefers-reduced-motion` media query for accessibility
- CSS `clamp()` for fluid typography (no JS resize handlers)

Step 5: Deployment

Build for production:

```
npm run build
```

Output goes to `dist/`. Serve with any static host:

- **GitHub Pages** — Push `dist/` to `gh-pages` branch
- **Vercel** — Connect repo, build command `npm run build`, output `dist`
- **Netlify** — Same as Vercel

Dependencies Summary

dependencies:

```
three                # 3D engine
@react-three/fiber    # React renderer for Three.js
@react-three/drei     # R3F helpers (Float, Stars, MeshDistort)
@react-three/postprocessing # Bloom & effects
react-icons           # Icon components
react-intersection-observer # Scroll detection
```

devDependencies:

```
@vitejs/plugin-react # Vite React plugin
vite                 # Build tool
```

Customization Tips

- **Colors** — Edit CSS variables in `:root` in `global.css`
- **Your projects** — Edit the `projects` array in `Projects.jsx`
- **Skills** — Edit the `skillCategories` array in `Skills.jsx`
- **Contact form** — Replace the Web3Forms `access_key` in `Contact.jsx`
- **Content** — All text is in the component files, just edit the strings

Tip: Take screenshots of your live projects using Chrome's headless mode:

```
chrome --headless=new --screenshot=public/images/project.png --window-size=1280,800 https://your-site.com
```

Made with ❤️ using React 18 + Three.js + Vite 5